Mojo 🔥

language, stdlib, and compiler

Billy Zhu · Lukas Hermann · Weiwei Chen
{billyzhu, lukas, weiwei.chen}@modular.com

Why Mojo?

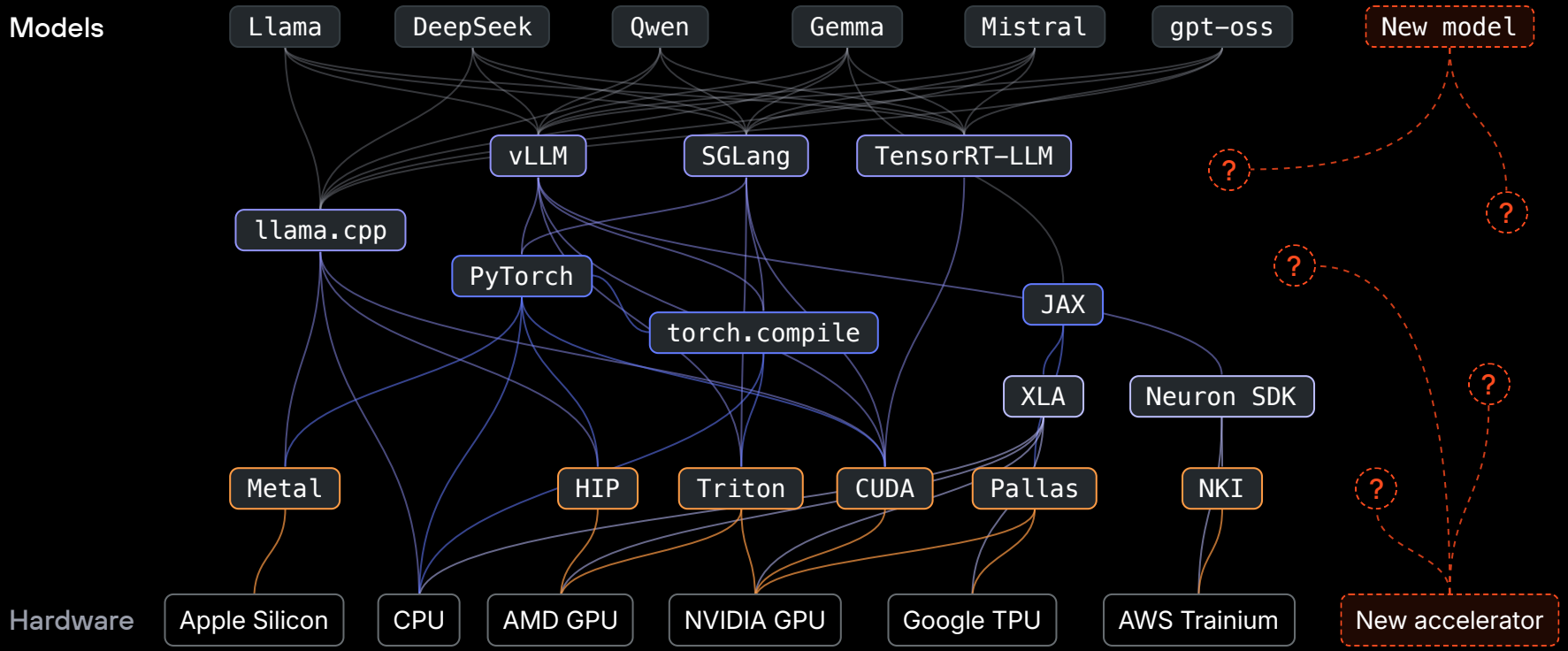
The problem it was built to solve, and why a new language was the answer



Modular's goal

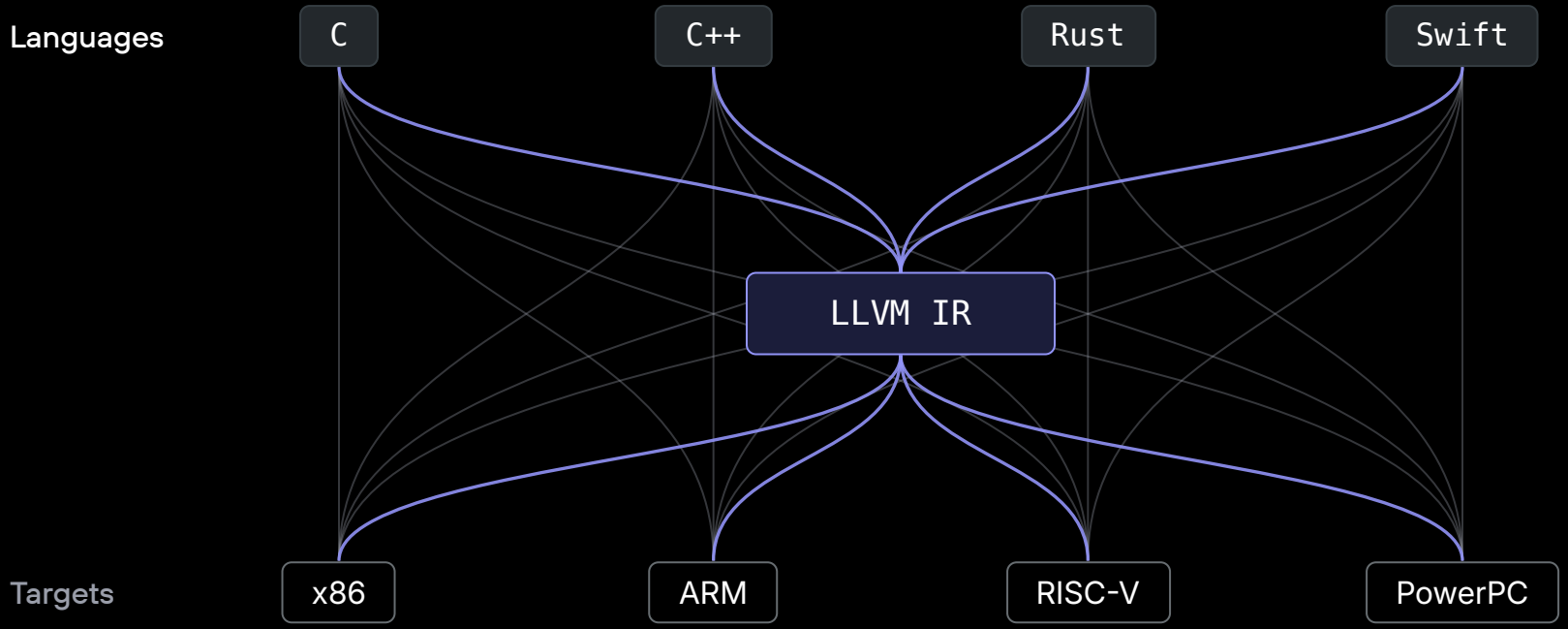
A unified inference stack that runs on heterogeneous hardware.

Why is this hard?

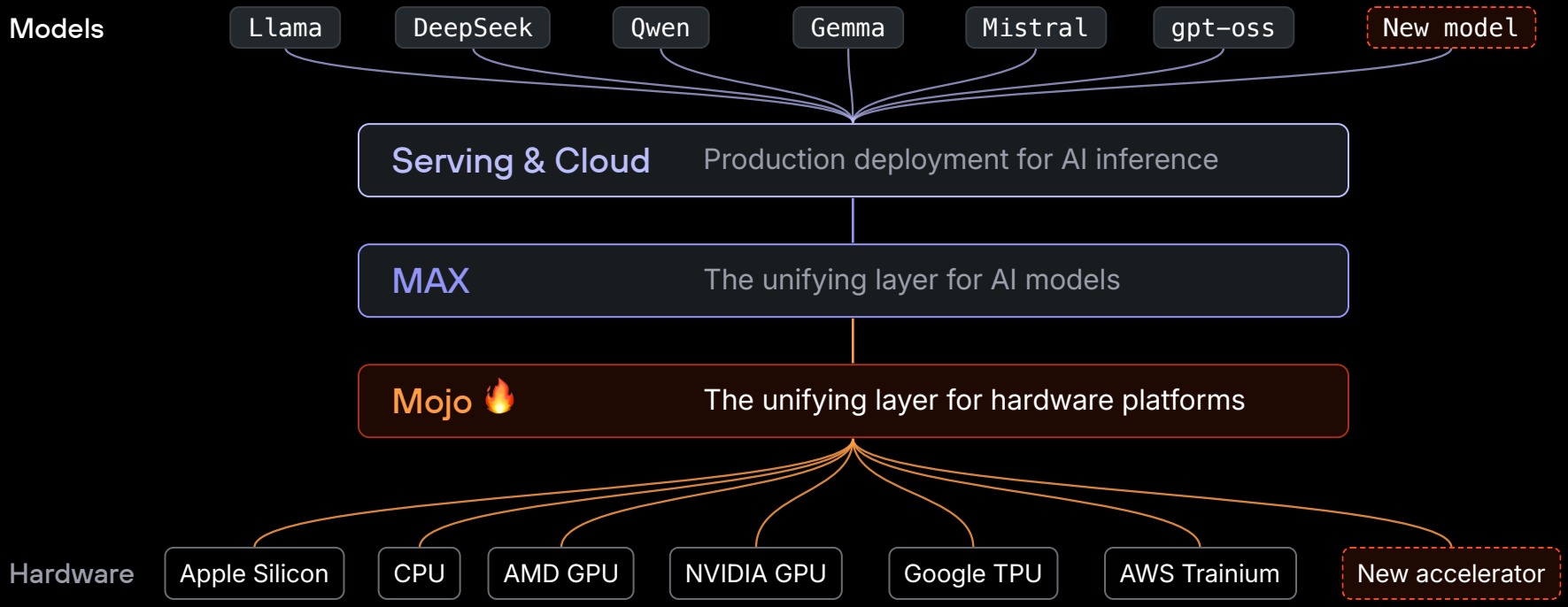


Incentives reward point solutions. The result is a web of fragmented systems.

The same shape of problem LLVM solved



Mojo unifies heterogeneous compute



Objective

Maximum performance on every hardware platform

Which means bridging different programming models.

Traditional compute

`LLVM IR`

`x86` `ARM` `RISC-V` `PowerPC`

Heterogeneous accelerators

No single programming model covers them all

`GPU` `TPU` `NPU` `Trainium` `…`

Every hardware ecosystem ships its own kernel language

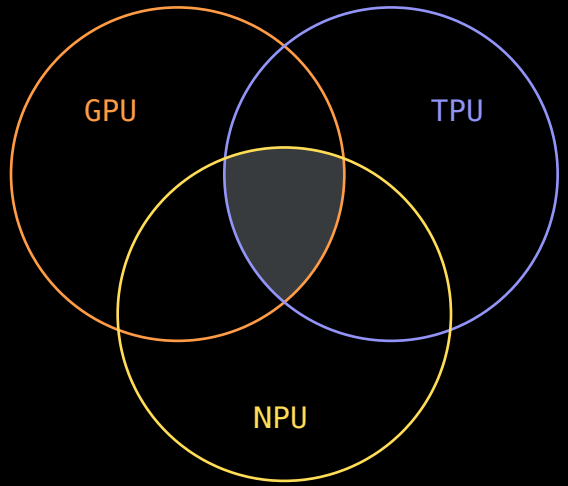
NVIDIA	AMD	Google TPU	AWS Trainium
`CUDA C++ / CuTe`	`HIP`	`Pallas`	`NKI`

A dual-sided moat: hard to leave, hard to enter.

This reinforces fragmentation.

How do we unify this?

What doesn’t work: the lowest common denominator



Uniformity at the expense of performance.
Target-specific capabilities stay hidden.

Getting performance back takes specialized knowledge:

Smart compilers

Find optimizations from each hardware’s characteristics

Vendor libraries

Optimized low-level libraries, per hardware

Every new operation needs a specialist

Smart compilers

A compiler pattern for every transformation, e.g. fusion

Vendor libraries

A target-specific implementation of every op

Key unlock

Performance knowledge must be programmable by domain experts.

Not sealed inside a compiler or a vendor library.

Modular's approach

A sufficiently ~~smart compiler~~ expressive language

Each hardware platform thrives via specialization, not a uniform mold.

Hardware knowledge lives in libraries, not the compiler.

MLIR provides half of the foundation

The common foundation for modern AI compilers

`IREE · Triton · torch-mlir · OpenXLA · tt-mlir · mlir-aie`

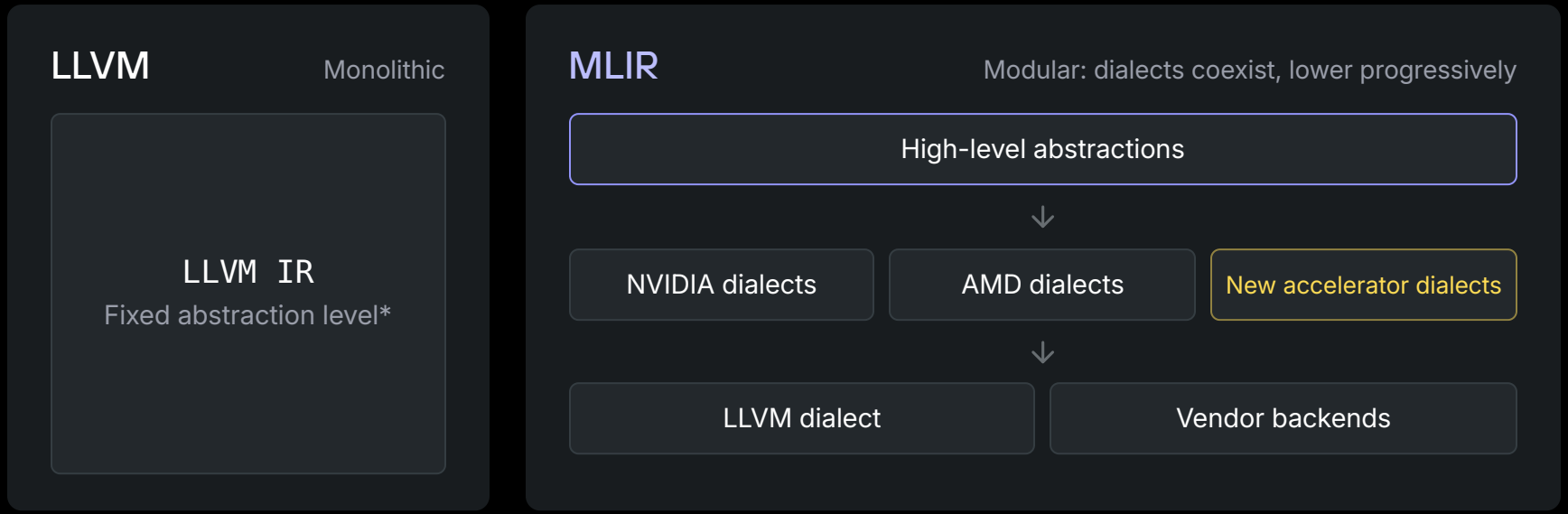
?

MLIR

`llvm dialect`
The LLVM programming model

`accelerator dialects`
Accelerators already use MLIR

Why MLIR worked where LLVM fell short



The natural substrate for unifying heterogeneous compute.

The missing half: Mojo, the user-facing layer

An opinionated, transparent wrapper over MLIR

Mojo 🔥

- Meta-programming over MLIR
  - Performant portability
- Safety, composition, diagnostics, tooling
  - Ease of use of a modern language

MLIR Mojo bottoms out here, not at a fixed set of builtin primitives.

The new landscape

01
A new accelerator

→

02
Exposes native ops as MLIR dialects, or via LLVM

→

03
Builds safe abstractions in Mojo libraries

→

04
Users keep one language and one toolchain

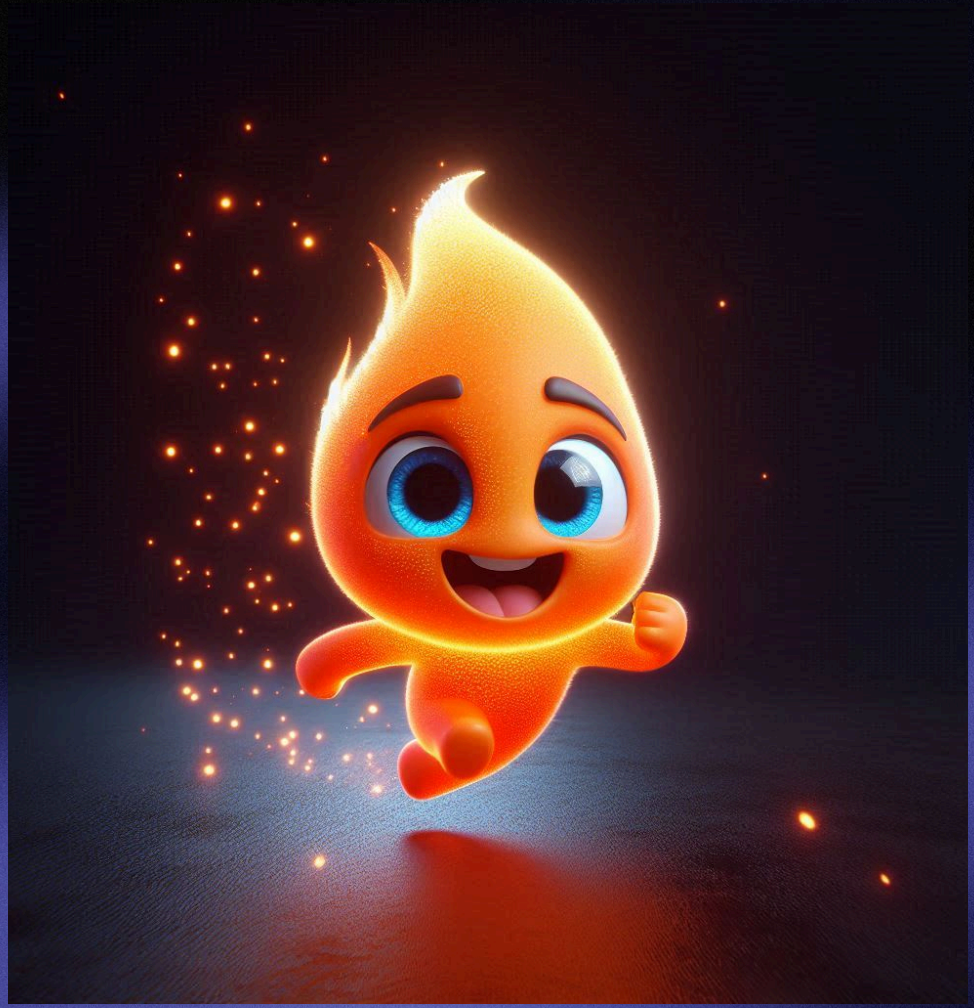
The problem	The answer
Uniform abstractions sacrifice target performance.	MLIR lets target-specific abstractions coexist.
Compiler-owned intelligence doesn't scale organizationally.	Mojo lets programmers build and compose them safely.

Up next

01 **Language**
The tools Mojo gives programmers

02 **Libraries**
Where hardware abstractions live

03 **Compiler**
How Mojo reaches MLIR, LLVM, and devices



Mojo in five minutes

A taste of the ideas that make Mojo distinctive

Python inherited syntax

```
struct Person:
    var name: String
    var favorite_number: Int32
    var date_of_birth: Tuple[Int, Int, Int]

def is_billy(p: Person) -> Bool:
    return p.name == "Billy"
```

Traits

```
trait Greetable:
    def greet(self):
        ...

struct Person(Greetable):
    var name: String
    var favorite_number: Int32
    var date_of_birth: Tuple[Int, Int, Int]

    def greet(self):
        print(t"Hello, {self.name}")

def use_greetable(g: Some[Greetable]):
    g.greet()
```

Mutable value semantics

Mojo

```
def by_immutable_reference(p: Person): ...
def by_mutable_reference(mut p: Person): ...
def parametrically_mutable_reference(ref p: Person): ...
def by_value(var p: Person): ...
```

C++

```
void by_immutable_reference(const Person& p) {}
void by_mutable_reference(Person& p) {}
template <typename P>
    requires std::same_as<std::remove_const_t<P>, Person>
void parametrically_mutable_reference(P& p) {}
void by_value(Person p) {}
```

Comptime: compile-time evaluation

```mojo
def sum(l: List[Int]) -> Int:
    var result = 0
    for elt in l:
        result += elt
    return result

# Does not allocate
def print_sum():
    comptime s = sum([1, 2, 3])
    print(s) # 6
    comptime s2 = sum([1, 2, 3, 4, 5])
    print(s2) # 15
```

```mojo
# branching disappears at runtime
def conditional():
    comptime s = sum([0, 0, 0])
    comptime if s == 0:
        print("zilch")
    else:
        print(s)
```




```mojo
# what ships in the binary
def conditional():
    print("zilch")
```

Origins: Mojo's borrow checker

```
def mutate(mut value: List[Int], mut value2: List[Int]):
    pass

def main():
    var l: List[Int] = [1, 2, 3]
    mutate(l, l)
    # error: aliasing values passed mutably to 'value' argument
    # and passed mutably to 'value2' argument in 'mutate' call
```

Parameters: generic programming

```
# Types and values can be parameters
struct Array[T: AnyType, length: Int]:
    # ...
    def __contains__(
        self, value: Self.T
    ) -> Bool where conforms_to(Self.T, Equatable):
        comptime for i in range(Self.length):
            if self[i] == value:
                return True
        return False

def contains_5(l: Array[Int, _]) -> Bool:
    return 5 in l
```

Library-defined abstractions

Where Mojo's core abstractions are defined, in Mojo itself



Every primitive lives in libraries

```
struct SIMD[dtype: DType, length: SIMDLength]:
    # ...

comptime Scalar = SIMD[
    _, length=__mlir_attr[`#lit.struct<{_mlir_value = 1}> : `, SIMDLength]
]

comptime Int = Scalar[DType.int]
comptime Int8 = Scalar[DType.int8]
comptime UInt8 = Scalar[DType.uint8]
comptime Int16 = Scalar[DType.int16]
comptime UInt16 = Scalar[DType.uint16]
comptime Int32 = Scalar[DType.int32]
comptime UInt32 = Scalar[DType.uint32]
comptime Int64 = Scalar[DType.int64]
comptime UInt64 = Scalar[DType.uint64]
comptime Int128 = Scalar[DType.int128]
comptime UInt128 = Scalar[DType.uint128]
comptime Int256 = Scalar[DType.int256]
comptime UInt256 = Scalar[DType.uint256]
```

LLVM composition

```
@inline(.always)
def iota[
    dtype: DType, width: Int
](offset: Scalar[dtype] = Scalar[dtype](0)) -> SIMD[dtype, width]:

    comptime step_dtype = dtype if dtype.is_integral() else DType.int

    var step: SIMD[step_dtype, width]
    if __is_run_in_comptime_interpreter:
        # ...
    else:
        # you can just call llvm inline!!
        step = llvm_intrinsic[
            "llvm.stepvector", SIMD[step_dtype, width], has_side_effect=False
        ]()
    return step.cast[dtype]() + offset
```

Accelerator abstractions

```
struct TileTensor[
    mut: Bool,
    //,
    dtype: DType,
    LayoutType: TensorLayout,
    origin: Origin[mut=mut],
    *,
    Engine: TensorEngine = DefaultEngine[element_width=1],
    address_space: AddressSpace = .GENERIC,
]:
    var _storage: Self.Engine.StorageType[
        Self.dtype, Self.origin, Self.address_space
    ]
    var layout: Self.LayoutType
```

```
trait TensorEngine:
    comptime StorageType[
        mut: Bool,
        //,
        dtype: DType,
        origin: Origin[mut=mut],
        address_space: AddressSpace,
    ]: TrivialRegisterPassable
```

TensorEngines define how to lower operations

```
trait TensorOps(TensorEngine):
    @staticmethod
    def iadd[
        # ...
    ](
        storage: Tuple[
            Self.StorageType[...],
            SelfLayoutType,
        ],
        other: Tuple[
            OtherEngine.StorageType[...],
            OtherLayoutType,
        ],
    ):
        __mlir_op.`my_dialect.add`(storage[0], other[0])
```

Which allows for high-level syntax

```
from layout import ImmTileTensor, MutTileTensor
from layout.tensor_engine import TensorOps

def kernel(
    var a: MutTileTensor[...],
    b: ImmTileTensor[dtype=a.dtype, Engine=a.Engine, ...]
) where conforms_to(a.Engine, TensorOps):
    a += b
```



Mojo compiler architecture

A Pythonic general-purpose language, and the “LLVM layer” for writing AI kernels

Mojo as the “LLVM layer” for GenAI

LLVM

C · C++ · Rust · Swift · Fortran

↓

LLVM IR 

One middle end

↓

x86 · ARM · RISC-V · PowerPC · NVPTX · AMDGPU

Mojo

frameworks · graph compilers · serving

↓

Mojo 🔥

Kernels + hardware abstractions

↓

NVIDIA · AMD · Apple · Qualcomm · TPU · Trainium

[Why ML Needs a New Programming Language — Chris Lattner, Jane Street]

Mojo compiler architecture

Mojo Parser 🔥

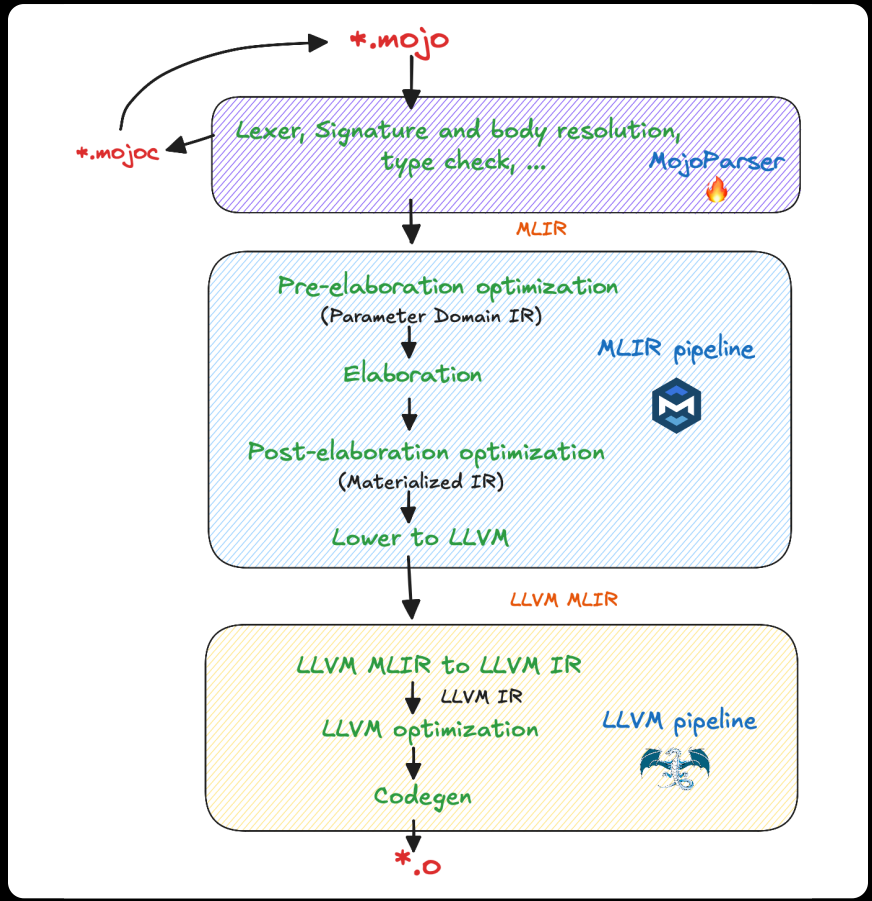
Platform agnostic, lazy parsing, parameter inference, constraints check (`where` clauses), ...

KGENCompiler MLIR layer

Check lifetimes, elaboration, IR optimizations, ...

ObjectCompiler LLVM layer

Lower MLIR to LLVM, LLVM optimization and code generation, ...



Mojo compiler built with both MLIR and LLVM

Mojo's MLIR dialects

`LIT`	parser dialect, close to source
`KGEN`	middle-level IR, "kernel generator"
`HLCF`	structured control flow, no gotos
`POP`	library-level ops: variadics, strings
`CO`	co-routines
`interp`	internally for the interpreter

Upstream MLIR, used sparingly: `index` and `LLVM`, plus `NVVM` and `ROCDL` for devices. No `arith`, `vector`, `affine`, `memref` or `linalg`.

MLIR framework: more modern and efficient

Flexible IR abstraction levels · built-in parallelization for passes · debugging and IR serialization support

Customized LLVM pipeline

- GVN · load/store optimization · LSR · scalar optimization · target codegen · inliner
- Vectorizer · loop unroller · coroutine passes · most inter-procedural passes

Parallelized LLVM compilation

1. Split subgraphs for optimization, call graphs intact
2. Split functions for parallel codegen
3. Relink in memory with MCLink

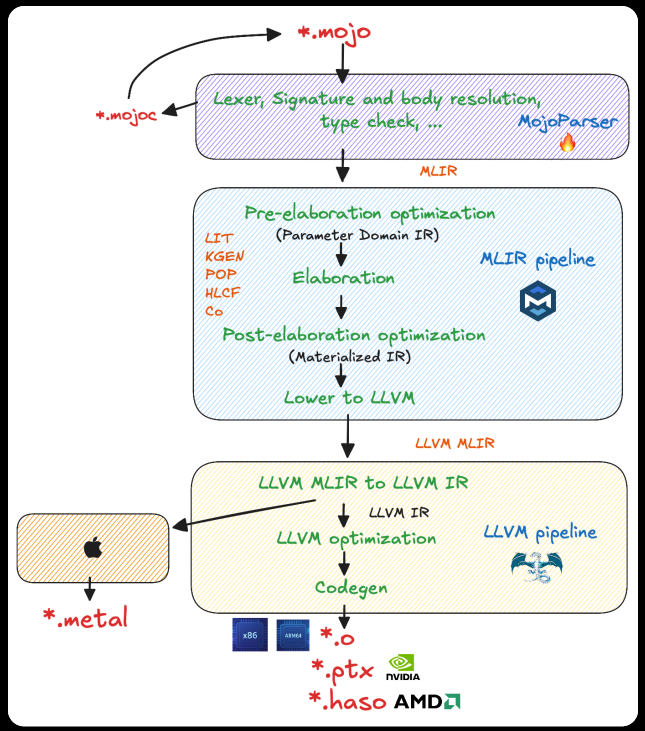
LLVM share of compile time

~~60–80%~~ → 20–30%

What we've learned from building Mojo's optimization pipeline

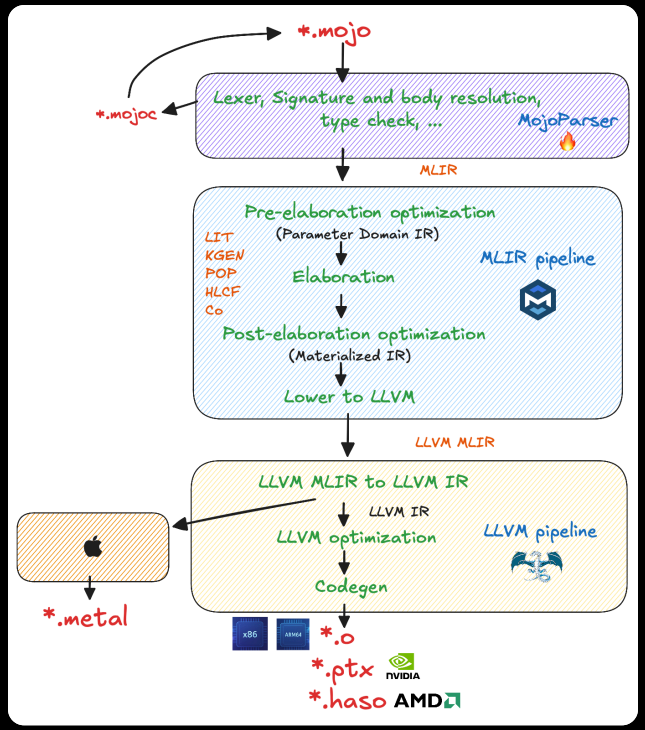
Mojo compilation to different targets

Targets with the LLVM backend

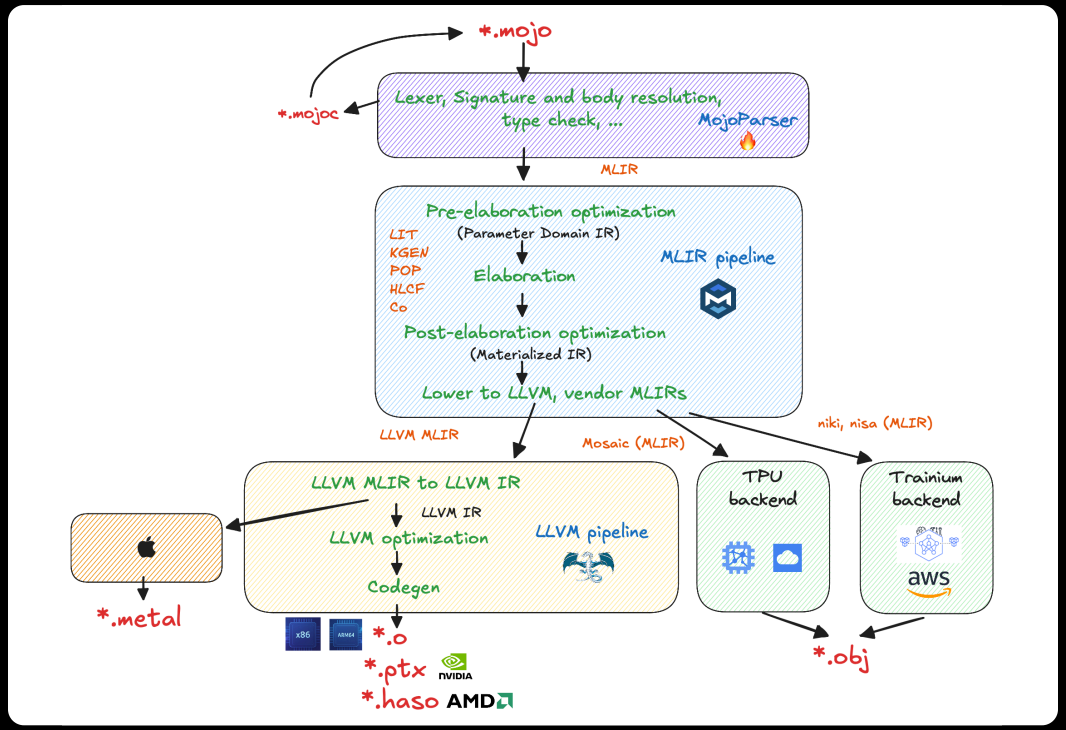


Mojo compilation to different targets

Targets with the LLVM backend



Targets with vendor MLIR backends



Mojo offload compilation

Host and device code in one language and one program module



Host and device code in one program

GPU kernel code

```
1  from gpu import global_idx
   from layout import Layout, LayoutTensor

2  comptime dtype = DType.float32
   comptime WIDTH = 10
   comptime BLOCK = 8
   comptime layout = Layout.row_major(WIDTH)

3  def vector_addition(
4      lhs: LayoutTensor[dtype, layout, MutableAnyOrigin],
       rhs: LayoutTensor[dtype, layout, MutableAnyOrigin],
       out: LayoutTensor[dtype, layout, MutableAnyOrigin],
       size: Int,
   ):
5      var tid = global_idx.x
       if tid < UInt(size):
           out[tid] = lhs[tid] + rhs[tid]
```

1 Library imports 2 Constants 3 GPU function
4 GPU tensor representation 5 GPU global offset

CPU host code

```
   def main():
1      constrained[has_accelerator(), "needs a GPU"]()
2      var ctx = DeviceContext()

3      var lhs_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
       var out_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
4      var lhs = LayoutTensor[dtype, layout](lhs_buf)
       var out = LayoutTensor[dtype, layout](out_buf)

5      ctx.enqueue_function_checked[vector_addition,
                                    vector_addition](
           lhs, rhs, out, WIDTH,
           grid_dim=ceildiv(WIDTH, BLOCK), block_dim=BLOCK,
       )

6      with out_buf.map_to_host() as host_buf:
           print(LayoutTensor[dtype, layout](host_buf))
```

1 Static assertion 2 GPU DeviceContext 3 GPU device buffers
4 GPU tensors 5 Compile and launch GPU kernel
6 Device buffer to host

Unified programming for CPU + GPU. One module, no separate device source, no vendor toolkit at build time. **GPU entry function as a parameter.**

Mojo offload compilation flow

```mojo
from gpu import global_idx
from layout import Layout, LayoutTensor

comptime dtype = DType.float32
comptime WIDTH = 10
comptime BLOCK = 8
comptime layout = Layout.row_major(WIDTH)

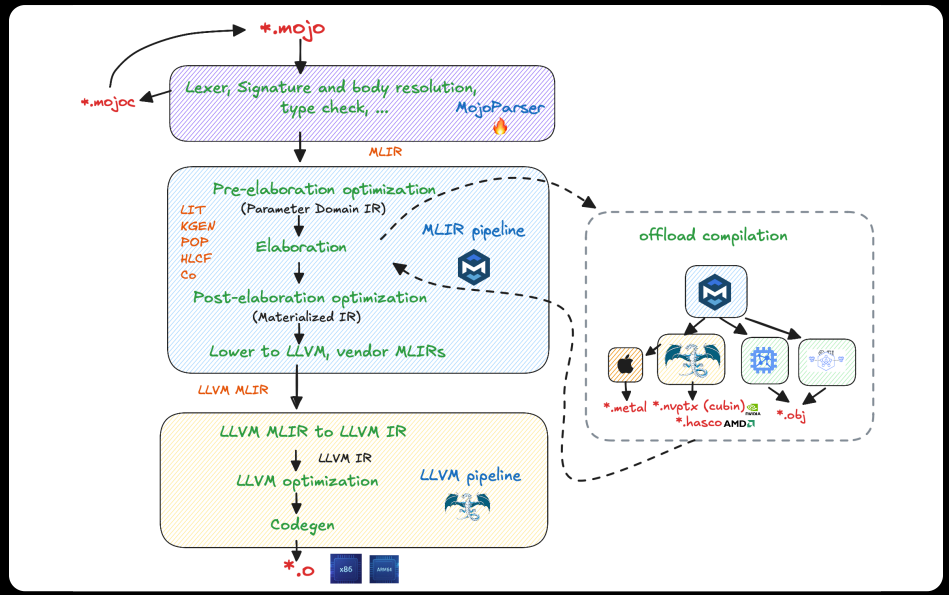
def vector_addition(
    lhs: LayoutTensor[dtype, layout, MutableAnyOrigin],
    rhs: LayoutTensor[dtype, layout, MutableAnyOrigin],
    out: LayoutTensor[dtype, layout, MutableAnyOrigin],
    size: Int,
):
    var tid = global_idx.x
    if tid < UInt(size):
        out[tid] = lhs[tid] + rhs[tid]

def main():
    constrained[has_accelerator(), "needs a GPU"]()
    var ctx = DeviceContext()

    var lhs_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    var out_buf = ctx.enqueue_create_buffer[dtype](WIDTH)
    var lhs = LayoutTensor[dtype, layout](lhs_buf)
    var out = LayoutTensor[dtype, layout](out_buf)

    ctx.enqueue_function_checked[vector_addition,
                                 vector_addition](
        lhs, rhs, out, WIDTH,
        grid_dim=ceildiv(WIDTH, BLOCK), block_dim=BLOCK,
    )

    with out_buf.map_to_host() as host_buf:
        print(LayoutTensor[dtype, layout](host_buf))
```



1. Offload during elaboration, like constant folding
2. Slice device code into its own MLIR module, compile to binary
3. Feed the binary back into the host module IR as a string

Not a frontend split. Not a runtime decision. Not a second compiler invocation.



The road ahead

The big picture, and where Mojo goes from here

The road ahead

Phase 1

High-performance kernel programming

Metaprogramming

Traits, constraints

Layout & tensor abstractions

Open source compiler

...

Phase 2

Systems application programming

Async

Pattern matching

Existentials / dynamic traits

Packaging and toolchain

...

Phase 3

Dynamic object-oriented programming

Untyped variables

Classes

Inheritance

...

Continuous investments

- Developer experience
- Performance & compile time
- Libraries and ecosystem
- Hardware reach

Contributing to Mojo

Mojo is fully open source on GitHub, and we're accepting contributions in several areas.

01
Compiler

02
Standard library

03
Docs

Each team accepts a specific set of changes, so check the contribution areas before you start work.

`github.com/modular/modular`

`WIP` Process for Qualcomm internal contributions to the Modular monorepo

Q&A